

Python — DICTIONARY (Part 1)

A dictionary stores data as KEY : VALUE pairs.

Mutable; preserves insertion order in modern Python.

Keys are unique and hashable; values may repeat.

Create with {} or dict().

1. Creating a Dictionary

```
student = {'name': 'Jahanur', 'age': 22, 'course': 'Python'}  
empty = {}  
empty2 = dict()
```

2. Keys and Values ☆

```
student['name'] # 'Jahanur'  
student['age']  # 22
```

3. Duplicate Keys

```
d = {'a': 10, 'a': 20}  
print(d) # {'a': 20}
```

4. Different Value Types

```
d = {'name': 'Jahanur', 'marks': [80, 90], 'active': True,  
     'address': {'city': 'Dinhata'}}
```

5. Access: [] and get()

```
d['name']  
d.get('age')  
d.get('salary') # None
```

Python — DICTIONARY (Part 2)

1. [] vs get() ☆

`d['missing']` raises `KeyError`.

`d.get('missing')` returns `None` or a default value.

```
d.get('city')
d.get('city', 'Unknown')
```

2. Add and Update

```
d = {'name': 'Jahanur'}
d['age'] = 22          # add
d['age'] = 23          # update
```

3. update() ☆

```
d.update({'b': 20, 'c': 3})
d.update(other_dict)
```

4. Removing Items

```
d.pop('key')
del d['key']
d.popitem()      # last inserted pair
d.clear()        # empty dictionary
```

5. copy()

```
d2 = d1.copy()
d2['x'] = 99     # d1 is unchanged
```

Python — DICTIONARY (Part 3)

1. Important Methods ☆

`get()` → safe lookup

`keys()` → all keys

`values()` → all values

`items()` → key-value pairs

`update()` → add/replace

`pop()` → remove by key

`popitem()` → remove last pair

`setdefault()` → get or insert

`copy()` → shallow copy

`clear()` → empty

2. `keys()`, `values()`, `items()`

```
d = {'a':10, 'b':20}
d.keys()
d.values()
d.items()
```

3. Loop Through a Dictionary ☆

```
for key in d: print(key)
for value in d.values(): print(value)
for key, value in d.items(): print(key, value)
```

4. Membership

`key in d` checks KEYS, not values.

```
'name' in d
'Jahanur' in d.values()
```

Python — DICTIONARY (Part 4)

1. Nested Dictionaries ☆

```
students = {'s1': {'name': 'Jahanur', 'age': 22},
            's2': {'name': 'Rahul', 'age': 21}}
students['s1']['name']
```

2. Dictionary with List Values

```
marks = {'Jahanur': [80, 90, 85]}
marks['Jahanur'][1]    # 90
```

3. Dictionary Comprehension [?]

```
squares = {x:x*x for x in range(1,6)}
even = {x:x for x in range(10) if x%2==0}
```

4. Conditional Comprehension

```
nums=[1,2,3,4,5,6]
r={x:x*x for x in nums if x%2==0}
```

5. fromkeys() and setdefault()

```
d = dict.fromkeys(['a', 'b', 'c'], 0)
d.setdefault('d', 100)
```

6. Dictionary Unpacking ☆

```
a={'x':1, 'y':2}; b={'z':3}
c={**a, **b}
c2={**a, 'y':99}
```

Python — DICTIONARY (Part 5)

1. Dictionary vs Other Collections

LIST → indexed, mutable, duplicates allowed

TUPLE → indexed, immutable, duplicates allowed

SET → unique, no indexing, mutable

DICT → key:value mapping, unique keys, mutable

2. Important Rules ☆

Keys must be hashable.

Keys must be unique.

Values can repeat and can be mutable.

[] may raise KeyError; get() is safe.

in checks dictionary keys.

3. Common Mistakes

```
d={'a':1}
d['b']          # KeyError
d.get('b')      # None
d.append(...)   # no append()
d['b']=2        # correct
```

4. DSA Connection ?

Dictionaries power frequency counting, hash maps, caching, grouping, and fast lookup.

Typical average-case lookup/insert/delete is $O(1)$.

5. Frequency Count

```
nums=[1,2,2,3,3,3]
freq={}
for x in nums: freq[x]=freq.get(x,0)+1
print(freq)    # {1:1,2:2,3:3}
```

6. Two Sum Pattern

```
seen={}
for i,x in enumerate(nums):
    need=target-x
    if need in seen: print(seen[need],i)
    seen[x]=i
```

Python — DICTIONARY (Part 6)

1. Beginner Practice

1. Create a student dictionary.
2. Add and update values.
3. Delete a key safely.
4. Print keys, values, and pairs.
5. Count key-value pairs.

2. Intermediate Practice

6. Character frequency.
7. Word frequency.
8. Key with largest value.
9. Merge dictionaries.
10. Invert a dictionary when values are unique.
11. Group words by first letter.

3. DSA Practice

12. Two Sum.
13. First non-repeating character.
14. Find duplicates.
15. Group anagrams.
16. Most frequent element.
17. Prefix-sum frequency problems.

4. Quick Revision ☆

DICT = {key:value}

KEY = unique + hashable

VALUE = may repeat

[] = direct access / *KeyError*

get() = safe access

update() = add/replace

pop() = remove

keys()/values()/items() = iterate

Comprehension = {*k:v* for ...}

Nested dictionaries = dictionaries inside dictionaries

5. Mini Challenge ?

Build a student marks system with 5 students.

Find highest scorer, average marks, and students above 80.

Solve it using dictionaries and loops.